

目录

1 任务一

- 1.1 作业要求
- 1.2 图表分析
- 1.3 程序设计
 - 1.3.1 设计思路
 - 1.3.2 核心代码
- 1.4 执行方法和运行结果
 - 1.4.1 执行方法
 - 1.4.2 运行结果

2 任务二

- 2.1 作业要求
- 2.2 图表分析
- 2.3 程序设计
 - 2.3.1 设计思路
 - 2.3.2 核心代码
- 2.4 执行方法和运行结果
 - 2.4.1 执行方法
 - 2.4.2 运行结果

3 任务三

- 3.1 作业要求
- 3.2 实验分析
- 3.3 程序设计
 - 3.3.1 设计思路
 - 3.3.2 核心代码
- 3.4 执行方法和运行结果
 - 3.4.1 执行方法
 - 3.4.2 运行结果

4 任务四

- 4.1 作业要求
- 4.2 图表分析
- 4.3 程序设计
 - 4.3.1 设计思路
 - 4.3.2 核心代码

4.4 执行方法和运行结果

4.4.1 执行方法

4.4.2 运行结果

5 任务五

5.1 作业要求

5.2 图表分析

5.3 程序设计

5.3.1 设计思路

5.3.2 核心代码

5.4 执行方法和运行结果

5.4.1 执行方法

5.4.2 运行结果

6 作业总结与心得体会

7 附录A：提交材料清单和说明

作业三 数据可视化

1 任务一

1.1 作业要求

依照图 1-1 绘制历次人口普查全国人口数量柱状图。要求：

1. 使用matplotlib绘图库进行绘图。
2. 绘图应使用标题、坐标轴标签、坐标轴刻度及标签、图例等绘图元素，完整、清晰地展示数据信息。
3. 以图绘图的作业中，不要求新绘图的线条样式、图例写法、字体大小、间距、颜色等格式与原图严格一致。
4. 报告中需包含完成每项工作的核心代码、绘制出的图形及必要的说明。

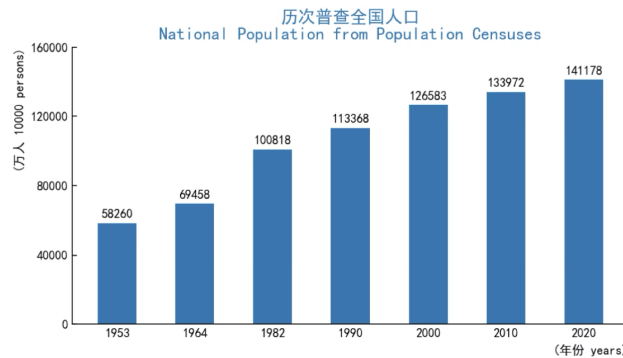


图 1-1 任务一

1.2 图表分析

通过分析图 1-1，可以得出以下关键信息：

1. **图表类型**：柱状图（Bar Chart），用于展示不同年份的人口数量对比。
2. **数据特征**：
 - X轴：年份（1953, 1964, 1982, 1990, 2000, 2010, 2020），共7个数据点如下（年份 - 人口数，单位：万人）：
 - 1953: 58260
 - 1964: 69458
 - 1982: 100818

- 1990: 113368
 - 2000: 126583
 - 2010: 133972
 - 2020: 141178
- Y轴：人口数量（单位：万人），范围从0到160000，刻度间隔为40000
 - 数据趋势：整体呈现上升趋势，从1953年的58260万人增长到2020年的141178万人

3. 图表元素：

- 标题：图表顶部有中英文标题
- 坐标轴标签：X轴标注"年份"，Y轴标注"人口数量（万人）"
- 数据标签：每个柱子顶部显示具体数值
- 图例：显示"全国人口数量"（样例未给出，但按照要求需要给出图例）
- 颜色：使用蓝色系柱状图

1.3 程序设计

1.3.1 设计思路

本程序使用 Python 的 matplotlib 库绘制柱状图，主要设计思路如下：

1. **字体设置**：配置中文字体支持，确保图表中的中文正常显示
2. **数据准备**：定义年份和对应的人口数量数据
3. **图表创建**：创建合适尺寸的图表对象
4. **柱状图绘制**：使用 `ax.bar()` 方法绘制柱状图
5. **图表元素添加**：依次添加标题、坐标轴标签、数据标签、图例等元素
6. **样式优化**：设置Y轴刻度范围和间隔，调整布局

1.3.2 核心代码

程序首先导入必要的库，包括 matplotlib 用于绘图，numpy 用于数值计算。由于图表中包含中文内容，需要设置中文字体支持。通过

`mpl.rcParams['font.family']` 设置字体为 'STHeiti'，并设置 `axes.unicode_minus` 为 False 以解决负号显示问题。

1. 字体和基础配置

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import matplotlib as mpl
4
5 # 设置中文字体, 确保图表中的中文正常显示
6 mpl.rcParams['font.family'] = 'STHeiti'
7 mpl.rcParams['axes.unicode_minus'] = False
```

数据准备阶段, 程序定义了7个年份和对应的人口数量数据。年份以字符串形式存储, 便于作为分类标签使用; 人口数量以列表形式存储, 单位为万人。这些数据将作为柱状图的 X 轴和 Y 轴数据。

2. 数据准备

```
1 # 准备数据
2 years = ['1953', '1964', '1982', '1990', '2000', '2010', '2020']
3 # 人口数量, 单位: 万人 (10000 persons)
4 population = [58260, 69458, 100818, 113368, 126583, 133972, 141178]
```

图表创建阶段, 程序使用 `plt.subplots()` 创建一个 10×6 英寸的图表对象, 返回 figure 和 axes 对象。然后使用 `ax.bar()` 方法绘制柱状图, 传入年份列表和人口数量列表, 设置颜色为蓝色系, 并添加标签用于后续图例显示。

`ax.bar()` 方法返回柱状图对象列表, 便于后续添加数据标签。

3. 创建图表和绘制柱状图

```
1 # 创建图表和坐标轴对象
2 fig, ax = plt.subplots(figsize=(10, 6))
3
4 # 绘制柱状图
5 bars = ax.bar(years, population, color='#4682B4', label='全国人口数量')
```

标题和坐标轴标签设置阶段, 程序使用 `ax.set_title()` 设置图表标题, 包含中英文双语标题, 并设置字体大小、粗细和颜色。使用 `ax.set_xlabel()` 和 `ax.set_ylabel()` 分别设置 X 轴和 Y 轴的标签, 同样包含中英文说明, 便于理解。

4. 添加标题和坐标轴标签

```

1 # 添加标题
2 ax.set_title('历次普查全国人口\nNational Population from Population Censuses',
3             fontsize=16,
4             fontweight='bold',
5             color='#336699')
6
7 # 添加坐标轴标签
8 ax.set_xlabel('年份 (years)', fontsize=12)
9 ax.set_ylabel('人口数量 (万人 10000 persons)', fontsize=12)

```

数据标签添加阶段，程序遍历每个柱状图对象，获取其高度和位置信息。使用 `ax.text()` 方法在每个柱子顶部添加文本标签，显示具体的人口数量。文本位置设置为柱子中心上方，并设置水平对齐为居中，垂直对齐为底部，确保标签清晰可见。

5. 添加数据标签

```

1 # 添加数据标签
2 for bar in bars:
3     height = bar.get_height()
4     ax.text(bar.get_x() + bar.get_width() / 2., height + 1000,
5            f'{height}',
6            ha='center', va='bottom', fontsize=10)

```

Y 轴刻度和图例设置阶段，程序使用 `ax.set_ylim()` 设置 Y 轴范围从 0 到 160000，使用 `ax.set_yticks()` 设置刻度间隔为 40000，确保刻度清晰且符合原图要求。使用 `ax.legend()` 添加图例，位置设置在左上角，显示"全国人口数量"标签。

6. 设置Y轴刻度和图例

```

1 # 设置Y轴刻度
2 ax.set_ylim(0, 160000)
3 ax.set_yticks(np.arange(0, 160001, 40000))
4
5 # 添加图例
6 ax.legend(loc='upper left', fontsize=10)

```

最后，程序使用 `plt.tight_layout()` 调整布局，防止标签重叠。使用 `plt.savefig()` 保存图表为 PNG 格式，设置分辨率为 300 DPI，并使用 `bbox_inches='tight'` 参数去除多余的空白边距。保存成功后输出提示信息，最后使用 `plt.show()` 显示图表。

7. 显示和保存图形

```
1 # 调整布局, 防止标签重叠
2 plt.tight_layout()
3
4 # 保存图形
5 plt.savefig('population_census.png', dpi=300, bbox_inches='tight')
6 print('图表已保存为 population_census.png')
7
8 # 显示图形
9 plt.show()
```

1.4 执行方法和运行结果

1.4.1 执行方法

1. 环境要求:

- Python 3.x
- matplotlib 库
- numpy 库

2. 运行步骤:

```
1 # 进入项目目录
2 cd task3/task3-1
3
4 # 运行程序
5 python main.py
```

3. 依赖安装:

```
1 pip install matplotlib numpy
```

或使用项目中的 [uv](#) 包管理器:

```
1 uv sync
```

1.4.2 运行结果

程序运行后会弹出窗口显示生成的柱状图，同时保存在本地，如图 1-2 所示：

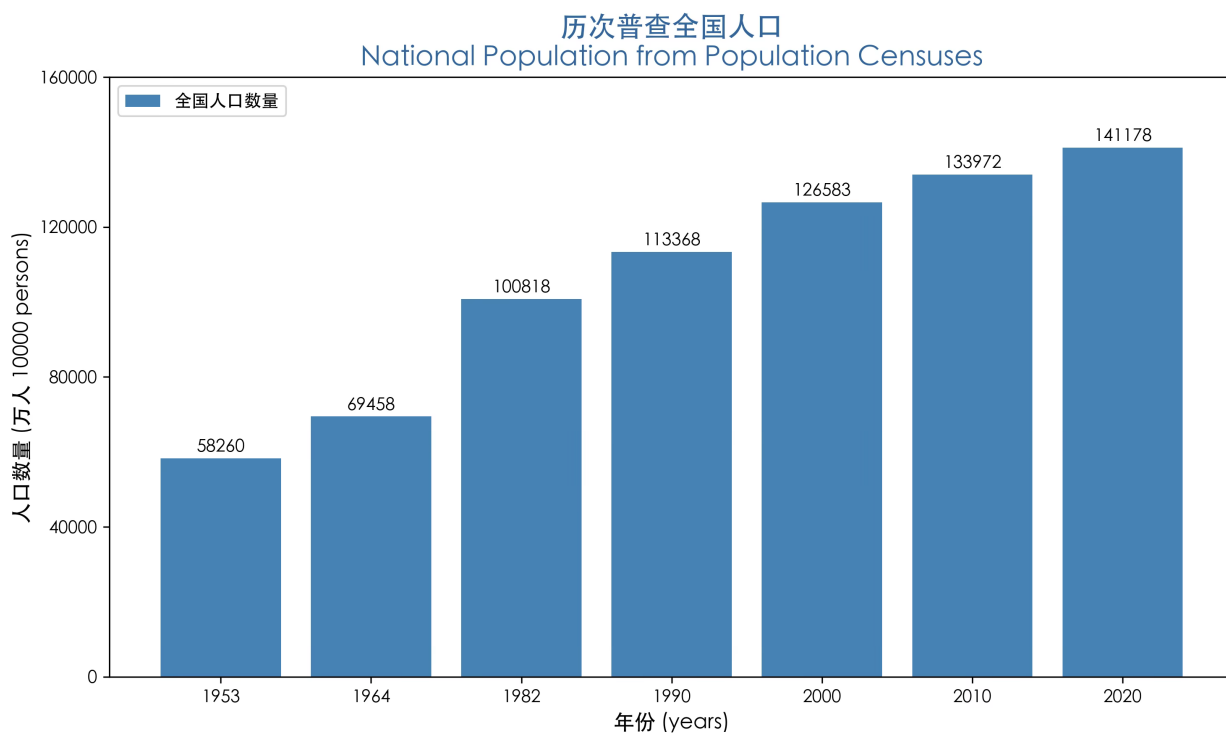


图 1-2 任务一复刻结果

图表正确包含原图或要求的以下特征：

- 图表标题：**顶部显示"历次普查全国人口"和"National Population from Population Censuses"双语标题
- 坐标轴：**
 - X轴：标注"年份 (years)"，显示7个年份标签（1953, 1964, 1982, 1990, 2000, 2010, 2020）
 - Y轴：标注"人口数量 (万人 10000 persons)"，刻度范围0-160000，间隔40000
- 柱状图：**
 - 7个蓝色柱状图，分别对应7个年份的人口数量
 - 每个柱子顶部显示具体数值（58260, 69458, 100818, 113368, 126583, 133972, 141178）
- 图例：**左上角显示"全国人口数量"图例

生成的图表完整、清晰地展示了历次人口普查的全国人口数量数据，符合作业要求的所有绘图元素标准。

2 任务二

2.1 作业要求

依据图 2-1 信息绘制某人 2020 年支付宝年支出情况饼图，图中应展示各类型支出占总支出的比例。其他要求同任务一。



图 2-1 任务二

2.2 图表分析

通过分析图 2-1，可以得出以下关键信息：

1. **图表类型：** 饼图（Pie Chart），用于展示各类支出占总支出的比例关系。
2. **数据特征：**
 - 共有8个支出分类，具体数据如下（分类 - 金额，单位：元）：
 - 酒店旅游: ¥21,914.00
 - 转账红包: ¥19,973.20
 - 餐饮美食: ¥10,379.59
 - 日用百货: ¥9,859.93
 - 交通出行: ¥8,351.35
 - 充值缴费: ¥2,428.54
 - 服饰装扮: ¥950.83
 - 互助保障: ¥827.20

- 总支出：¥83,684.64

3. 图表元素：

- 标题：图表顶部显示"2020年年度支出分类占比"和总支出金额
- 百分比标签：每个扇区显示对应的百分比数值
- 图例：显示各分类名称和对应金额，通常位于图表右侧
- 视觉效果：使用不同颜色区分各扇区，可能有阴影效果增强立体感
- 突出显示：最大的支出项（酒店旅游）可能被轻微分离以突出显示

4. 设计要点：

- 饼图清晰展示各类支出的占比关系
- 百分比标签便于直观了解各部分占比
- 图例提供详细的分类名称和金额信息
- 合理的颜色搭配确保各扇区易于区分

2.3 程序设计

2.3.1 设计思路

本程序使用 Python 的 matplotlib 库绘制饼图，主要设计思路如下：

1. **字体设置**：配置中文字体支持，确保图表中的中文正常显示
2. **数据准备**：定义8个支出分类和对应的金额数据，计算总支出
3. **图表创建**：创建合适尺寸的图表对象
4. **饼图绘制**：使用 `ax.pie()` 方法绘制饼图，设置百分比显示、起始角度、分离效果等
5. **图例添加**：创建包含分类名称和金额的图例标签，放置在图表右侧
6. **样式优化**：调整百分比标签样式，添加标题，确保饼图为圆形

2.3.2 核心代码

程序首先导入必要的库并设置中文字体支持，与任务一类似。数据准备阶段，程序定义了8个支出分类和对应的金额数据，使用 numpy 数组存储金额数据，便于后续计算。程序计算总支出金额，用于在标题中显示。

1. 字体和基础配置

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3 import matplotlib as mpl
4
5 # 设置中文字体
6 mpl.rcParams['font.family'] = 'STHeiti'
7 mpl.rcParams['axes.unicode_minus'] = False

```

数据准备阶段，程序定义了8个支出分类名称和对应的金额数据。使用 numpy 数组存储金额，便于后续计算总支出。程序计算总支出金额，用于在标题中显示。

2. 数据准备

```

1 # 准备数据
2 categories = [
3     '酒店旅游', '转账红包', '餐饮美食', '日用百货',
4     '交通出行', '充值缴费', '服饰装扮', '互助保障'
5 ]
6 expenses = np.array([
7     21914.00, 19973.20, 10379.59, 9859.93,
8     8351.35, 2428.54, 950.83, 827.20
9 ])
10
11 total_expense = expenses.sum()

```

绘图参数准备阶段，程序创建 `explode` 列表用于突出显示最大的支出项。列表的第一个元素为 0.05，表示将酒店旅游扇区分离 5%，其余7个元素为 0，表示不分离。这种设计能够吸引读者注意最大的支出项。

3. 准备绘图参数

```

1 # 突出显示最大的支出项（酒店旅游）
2 explode = [0.05] + [0] * 7

```

饼图绘制阶段，程序创建 10×8 英寸的图表对象，然后使用 `ax.pie()` 方法绘制饼图。该方法接收金额数组，设置 `labels=None` 避免在饼图上显示标签造成重叠，设置 `autopct='%1.1f%%'` 自动在每个扇区显示百分比（保留一位小数）。设置 `startangle=90` 使饼图从顶部开始，`explode` 参数实现分离效果，`shadow=True` 添加阴影增强立体感，`wedgeprops` 参数设置扇区边缘为白色，线宽为 1.5，增强扇区之间的区分度。方法返回三个对象：`wedges`（扇区对象）、`texts`（文本对象，本例中为空）、`autotexts`（百分比文本对象）。

4. 创建图表和绘制饼图

```

1  # 创建图表和坐标轴对象
2  fig, ax = plt.subplots(figsize=(10, 8))
3
4  # 绘制饼图
5  wedges, texts, autotexts = ax.pie(
6      expenses,
7      labels=None, # 不在饼图上显示标签, 避免重叠
8      autopct='%1.1f%%', # 显示百分比, 保留一位小数
9      startangle=90, # 起始角度
10     explode=explode, # 分离效果
11     shadow=True, # 阴影效果
12     wedgeprops={'edgecolor': 'white', 'linewidth': 1.5} # 扇区分割线
13 )

```

图例创建阶段，程序使用列表推导式创建图例标签，每个标签包含分类名称和对应的金额（格式化为两位小数）。然后使用 `ax.legend()` 方法添加图例，将图例放置在图表右侧外部。`bbox_to_anchor` 参数控制图例的位置，设置为 `(1, 0, 0.5, 1)` 表示图例位于图表右侧，垂直居中。

5. 创建图例

```

1  # 创建图例标签: 包含分类名称和具体金额
2  legend_labels = [f'{cat} (¥{exp:.2f})' for cat, exp in zip(categories,
3      expenses)]
4
5  # 添加图例, 放置在图表右侧
6  ax.legend(
7      wedges,
8      legend_labels,
9      title="支出分类 (金额)",
10     loc="center left",
11     bbox_to_anchor=(1, 0, 0.5, 1) # 放置在图表外部右侧
12 )

```

百分比标签样式调整阶段，程序遍历 `autotexts` 列表中的每个百分比文本对象，设置文本颜色为白色（确保在深色扇区上清晰可见），字体大小为 9，字体粗细为粗体，提高可读性。

6. 调整百分比标签样式

```

1  # 调整百分比标签的样式
2  for text in autotexts:
3      text.set_color('white')
4      text.set_fontsize(9)
5      text.set_fontweight('bold')

```

最后，程序设置图表标题，包含年份和总支出金额信息。使用 `ax.axis('equal')` 确保饼图为圆形（而非椭圆形）。调整布局后保存图表，设置分辨率为 300 DPI，最后显示图表。

7. 添加标题和保存图形

```
1 # 添加标题
2 ax.set_title(f'2020年年度支出分类占比\n(总支出: ¥{total_expense:.2f})',
3             fontsize=16,
4             fontweight='bold')
5
6 # 确保饼图是圆形
7 ax.axis('equal')
8
9 # 调整布局
10 plt.tight_layout()
11
12 # 保存图形
13 plt.savefig('expense_pie_chart.png', dpi=300, bbox_inches='tight')
14 print('图表已保存为 expense_pie_chart.png')
15
16 # 显示图形
17 plt.show()
```

2.4 执行方法和运行结果

2.4.1 执行方法

1. 环境要求：

- Python 3.x
- matplotlib 库
- numpy 库

2. 运行步骤：

```
1 # 进入项目目录
2 cd task3/task3-2
3
4 # 运行程序
5 python main.py
```

3. 依赖安装：

```
1 pip install matplotlib numpy
```

或使用项目中的 `uv` 包管理器：

```
1 uv sync
```

2.4.2 运行结果

程序运行后会弹出窗口显示生成的饼图，同时保存在本地，如图 2-2 所示：

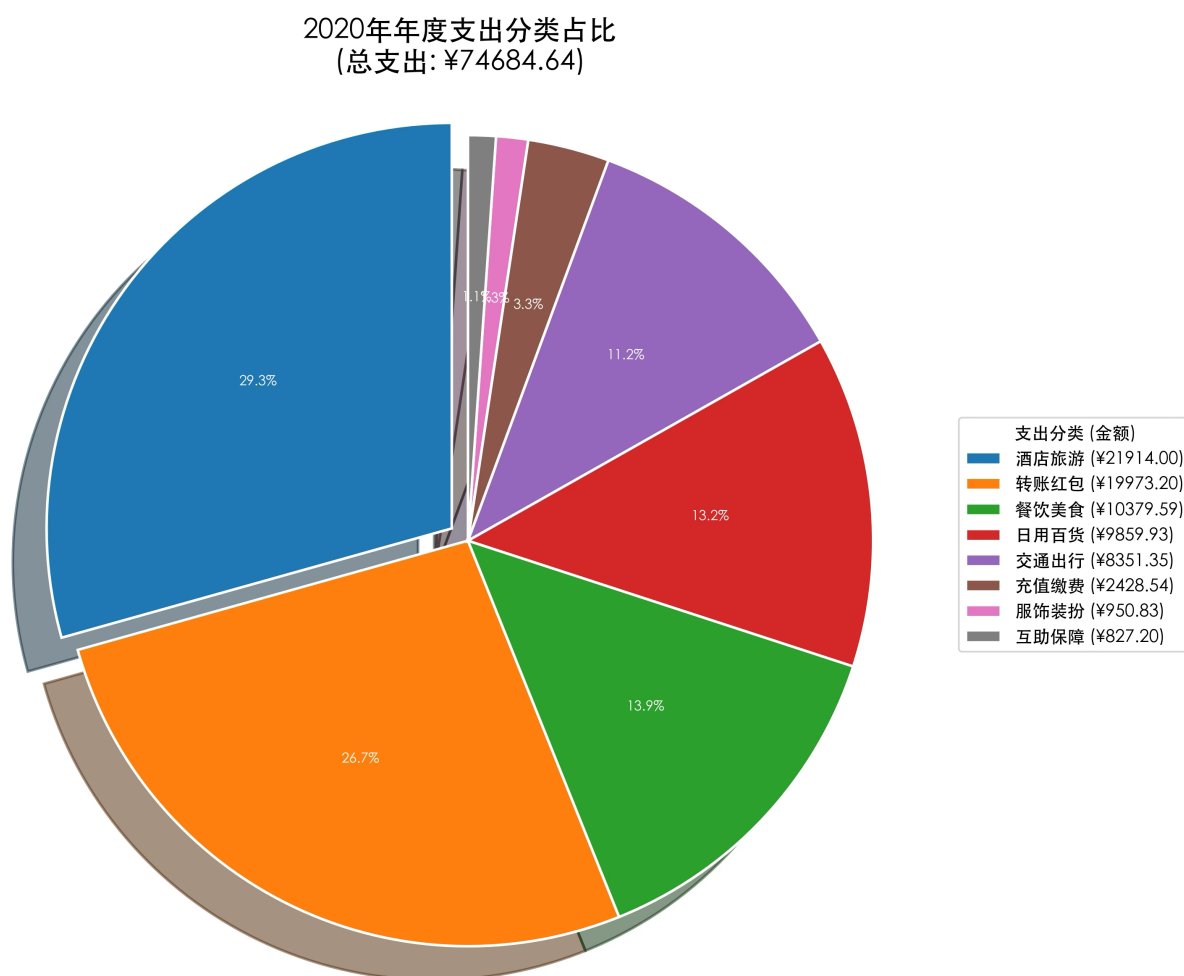


图 2-2 任务二复刻结果

图表正确包含原图或要求的以下特征：

1. **图表标题：**顶部显示"2020年年度支出分类占比"和总支出金额 (¥83,684.64)
2. **饼图扇区：**
 - 8个不同颜色的扇区，分别对应8个支出分类
 - 每个扇区显示对应的百分比数值（保留一位小数）
 - 最大的支出项（酒店旅游）被轻微分离以突出显示
 - 扇区之间有白色分割线，增强视觉效果
3. **图例：**
 - 位于图表右侧，显示所有支出分类

- 每个图例项包含分类名称和具体金额
- 图例标题为"支出分类(金额)"

4. 数据展示:

- 酒店旅游占比最高（约26.2%），金额为¥21,914.00
- 转账红包次之（约23.9%），金额为¥19,973.20
- 其他各项支出按金额从高到低依次排列
- 百分比标签清晰可见，便于读取各部分的占比

生成的图表完整、清晰地展示了2020年支付宝年度支出的分类占比情况，符合作业要求的所有绘图元素标准。

3 任务三

3.1 作业要求

使用 IRIS 数据集，在一个 figure 中绘制出如图 3-1 所示的 16 个子图。分别使用花瓣长度、花瓣宽度、花萼长度和花萼宽度这四种数据，两两组合绘制散点图。其他要求同任务一。

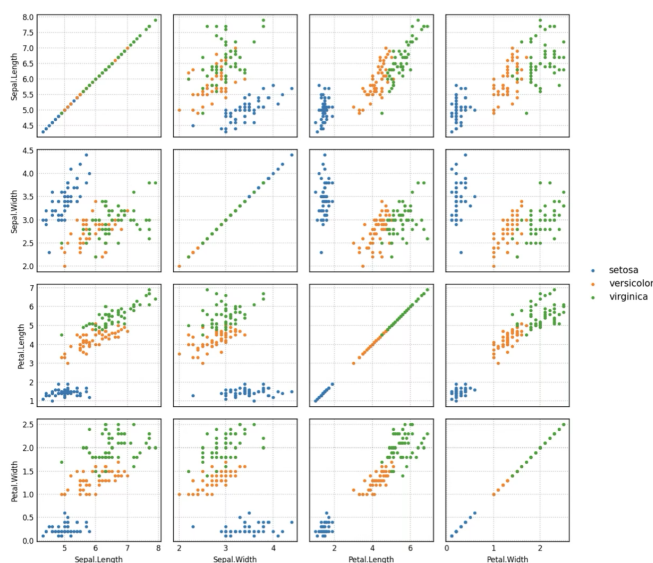


图 3-1 任务三

3.2 实验分析

通过分析图 3-1 和数据集，可以得出以下关键信息：

1. **图表类型：** 散点图矩阵（Scatter Plot Matrix），用于展示多个特征之间的两两关系。
2. **数据特征：**
 - IRIS 数据集包含 150 个样本，分为 3 个物种（setosa、versicolor、virginica）
 - 4 个特征：花萼长度（sepal length）、花萼宽度（sepal width）、花瓣长度（petal length）、花瓣宽度（petal width）
 - 每个特征的单位为厘米（cm）
3. **图表结构：**
 - 4×4 的子图矩阵，共 16 个子图

- 行和列分别对应 4 个特征

4. 图表元素：

- 标题：示例图没有，补充总标题 "IRIS Dataset Pairwise Scatter Matrix"
- 坐标轴标签：最底行显示 X 轴特征名称，最左列显示 Y 轴特征名称
- 刻度标签：内部子图隐藏刻度标签，避免信息冗余
- 图例：显示 3 个物种，用不同颜色区分（蓝色、橙色、绿色）
- 网格线：每个子图都有网格线，便于读取数值
- 颜色编码：不同物种使用不同颜色，便于区分

3.3 程序设计

3.3.1 设计思路

本程序使用 Python 的 matplotlib 库绘制散点图矩阵，主要设计思路如下：

1. **数据加载**：从 CSV 文件加载 IRIS 数据集，并进行数据预处理
2. **特征映射**：定义 4 个特征及其对应的列名映射关系
3. **子图创建**：创建 4x4 的子图矩阵，设置合适的尺寸和间距
4. **循环绘制**：使用双重循环遍历所有特征组合，绘制对应的散点图
5. **样式设置**：设置坐标轴标签、隐藏内部刻度、添加网格线
6. **图例添加**：在图表右侧添加图例，显示 3 个物种
7. **布局优化**：调整布局，确保所有元素清晰可见

3.3.2 核心代码

程序首先导入必要的库，包括 pandas 用于数据处理，matplotlib 用于绘图。数据加载阶段，程序使用 `os.path.join()` 构建 CSV 文件路径，确保路径在不同操作系统上都能正常工作。读取 CSV 文件后，程序将列名转换为小写，并使用 `rename()` 方法将 CSV 文件中的列名（如 'Sepal.Length'）映射为标准的 Python 命名格式（如 'sepal_length'），便于后续代码访问。

1. 字体和基础配置

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import pandas as pd
4 import matplotlib as mpl
5 import os
6
7 # 设置中文字体
8 mpl.rcParams['font.family'] = 'STHeiti'
9 mpl.rcParams['axes.unicode_minus'] = False
```

数据加载阶段，程序从 CSV 文件读取 IRIS 数据集。由于 CSV 文件的列名格式（如 'Sepal.Length'）与 Python 变量命名规范不一致，程序首先将列名转换为小写，然后使用字典映射将点号分隔的列名转换为下划线分隔的标准格式，同时将 'species' 列重命名为 'species_name'，便于后续处理。

2. 从 CSV 文件加载数据

```
1 # 从 CSV 文件加载 IRIS 数据集
2 csv_path = os.path.join(os.path.dirname(__file__), 'data', 'iris.csv')
3 df = pd.read_csv(csv_path)
4
5 # 将列名转换为小写，并映射到标准特征名称
6 df.columns = df.columns.str.lower()
7 # 重名列以匹配标准格式
8 df = df.rename(columns={
9     'sepal.length': 'sepal_length',
10    'sepal.width': 'sepal_width',
11    'petal.length': 'petal_length',
12    'petal.width': 'petal_width',
13    'species': 'species_name'
14 })
```

特征和物种信息定义阶段，程序定义了4个特征的显示名称（用于坐标轴标签）和对应的 DataFrame 列名。创建特征映射字典，将显示名称映射到列名，便于在循环中访问数据。程序从 DataFrame 中提取唯一的物种名称，并为每个物种分配颜色，创建物种到颜色的映射字典。

3. 定义特征和物种信息


```

17         # 非主对角线：绘制特征 feature_x vs feature_y 的散点图
18         feature_x_col = feature_map[feature_x]
19         feature_y_col = feature_map[feature_y]
20         for species_name in species_names:
21             subset = df[df['species_name'] == species_name]
22             ax.scatter(subset[feature_x_col], subset[feature_y_col],
23                       s=10, color=species_map[species_name],
24                       label=species_name)

```

坐标轴标签和样式设置阶段，程序根据子图位置决定是否显示刻度标签。对于非最底行的子图（ $i < 3$ ），隐藏 X 轴刻度标签；对于非最左列的子图（ $j > 0$ ），隐藏 Y 轴刻度标签，避免信息冗余。仅在最底行设置 X 轴标签，在最左列设置 Y 轴标签。标签文本通过字符串处理去除单位并替换空格为点号，符合原图格式。为每个子图添加虚线网格，便于读取数值。

6. 设置坐标轴标签和样式

```

1  # 隐藏刻度标签
2  if i < 3:
3      ax.set_xticklabels([])
4  if j > 0:
5      ax.set_yticklabels([])
6
7  # 设置轴标签（仅在最外层设置）
8  # x轴标签：只在最底行设置
9  if i == 3:
10     ax.set_xlabel(feature_x.split(' (cm)')[0].replace(' ', '.'), fontsize=10)
11     ax.tick_params(axis='x', rotation=90)
12  # y轴标签：只在最左列设置
13  if j == 0:
14     ax.set_ylabel(feature_y.split(' (cm)')[0].replace(' ', '.'), fontsize=10)
15
16  # 添加网格线
17  ax.grid(True, linestyle='--', alpha=0.6)

```

图例和保存阶段，程序从第一个子图（`axes[0, 1]`）中提取图例的 `handles` 和 `labels`，然后使用 `fig.legend()` 在图表级别添加图例，放置在图表右侧外部。使用 `plt.suptitle()` 添加总标题，位置在图表顶部。使用 `plt.tight_layout()` 调整布局，`rect` 参数为图例留出空间。最后保存和显示图表。

7. 添加图例和保存图形

```

1  # 添加图例
2  handles, labels = axes[0, 1].get_legend_handles_labels()
3  fig.legend(handles, labels,
4             loc='center right',
5             title="物种",

```

```
6         bbox_to_anchor=(1.05, 0.5),
7         fontsize=10)
8
9     # 添加总标题
10    plt.suptitle('IRIS Dataset Pairwise Scatter Matrix', y=1.02, fontsize=16)
11    plt.tight_layout(rect=[0, 0, 1.0, 1])
12
13    # 保存图形
14    plt.savefig('iris_scatter_matrix.png', dpi=300, bbox_inches='tight')
15    print('图表已保存为 iris_scatter_matrix.png')
16
17    # 显示图形
18    plt.show()
```

3.4 执行方法和运行结果

3.4.1 执行方法

1. 环境要求:

- Python 3.x
- matplotlib 库
- numpy 库
- pandas 库

2. 运行步骤:

```
1 # 进入项目目录
2 cd task3/task3-3
3
4 # 运行程序
5 python main.py
```

3. 依赖安装:

```
1 pip install matplotlib numpy pandas
```

或使用项目中的 `uv` 包管理器:

```
1 uv sync
```

3.4.2 运行结果

程序运行后会弹出窗口显示生成的散点图矩阵，同时保存在本地，如图 3-2 所示:

IRIS Dataset Pairwise Scatter Matrix

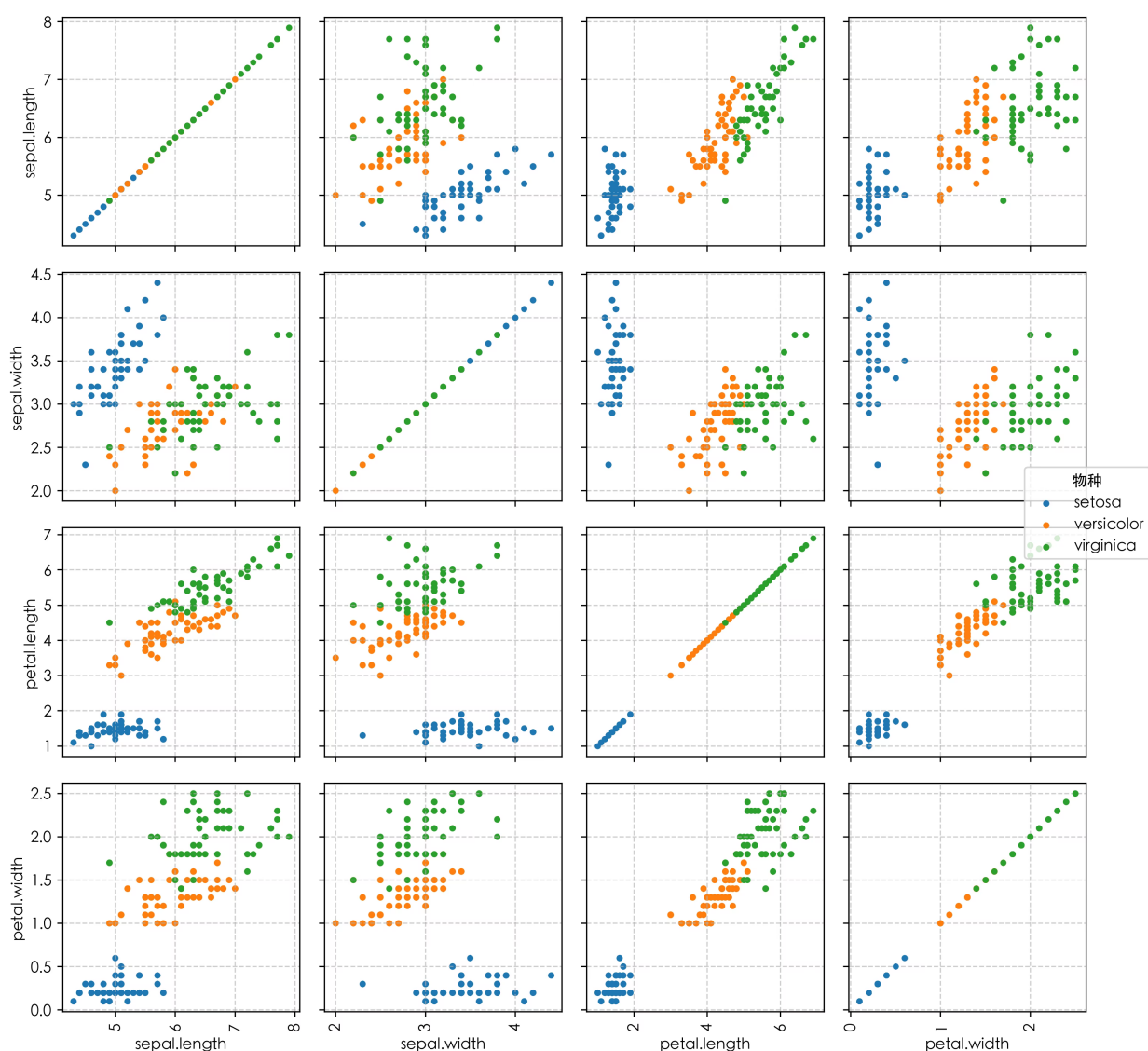


图 3-2 任务三复刻结果

图表正确包含原图或要求的以下特征：

1. 图表结构：

- 4x4 的子图矩阵，共 16 个子图
- 每个子图展示两个特征之间的关系

2. 主对角线子图：

- 显示特征与自身的关系，表现为一条直线上的散点
- 可以直观看出每个特征的数值分布范围

3. 非主对角线子图：

- 展示不同特征之间的散点关系
- 可以观察特征间的相关性和不同物种的分布模式
- 例如：花瓣长度与花瓣宽度呈现明显的正相关关系

4. 坐标轴标签：

- 最底行显示 X 轴特征名称 (sepal.length, sepal.width, petal.length, petal.width)
- 最左列显示 Y 轴特征名称
- 内部子图的刻度标签已隐藏，避免信息冗余

5. 图例：

- 位于图表右侧，显示 3 个物种 (setosa、versicolor、virginica)
- 使用不同颜色区分 (蓝色、橙色、绿色)

6. 网格线：

- 每个子图都有虚线网格，便于读取数值

生成的图表完整、清晰地展示了 IRIS 数据集中各特征之间的两两关系，符合作业要求的所有绘图元素标准。

4 任务四

4.1 作业要求

使用给出的"八年级期末考试成绩表.xlsx"，在一个 figure 中绘制六个子图，分别绘制六门课程的成绩分段统计情况直方图，每 10 分一个分段。其他要求同任务一。

4.2 图表分析

通过分析任务要求和数据集，可以得出以下关键信息：

1. **图表类型：**直方图（Histogram），用于展示成绩分布情况。
2. **数据特征：**
 - 数据来源：八年级期末考试成绩表
 - 六门课程：语文、数学、英语、物理、生物、地理
 - 分段方式：每10分一个分段（0-9, 10-19, 20-29, ..., 110+）
 - 统计内容：每个分段的学生人数
3. **图表结构：**
 - 2×3 的子图布局，共 6 个子图
 - 每个子图对应一门课程的成绩分布
 - 使用直方图展示各分数段的学生人数
4. **图表元素：**
 - 标题：每个子图显示课程名称和"成绩分段统计"
 - 坐标轴标签：X轴标注"分数分段"，Y轴标注"学生人数"
 - 刻度标签：X轴显示分段标签（0-9, 10-19, ...），Y轴显示人数
 - 数据标签：每个柱子上方显示具体的学生人数
 - 图例：显示"学生人数"
 - 网格线：Y轴方向添加网格线，便于读取数值

4.3 程序设计

4.3.1 设计思路

本程序使用 Python 的 matplotlib 库绘制成绩分段统计直方图，主要设计思路如下：

1. **数据加载**: 从 Excel 文件加载成绩数据
2. **分段设置**: 定义每10分一个分段, 从0到120分
3. **子图创建**: 创建 2×3 的子图布局
4. **循环绘制**: 遍历六门课程, 为每门课程绘制直方图
5. **样式设置**: 设置标题、坐标轴标签、刻度、数据标签等
6. **布局优化**: 调整布局, 确保所有元素清晰可见

4.3.2 核心代码

程序首先导入必要的库, 包括 pandas 用于读取 Excel 文件, matplotlib 用于绘图。数据加载阶段, 程序使用 `os.path.join()` 构建 Excel 文件路径, 使用 `pd.read_excel()` 读取第一个工作表的数据。程序定义了六门课程的列名, 这些列名将用于后续循环处理。

1. 字体和基础配置

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3 import numpy as np
4 import matplotlib as mpl
5 import os
6
7 # 设置中文字体
8 mpl.rcParams['font.family'] = 'STHeiti'
9 mpl.rcParams['axes.unicode_minus'] = False
```

数据加载阶段, 程序从 Excel 文件读取成绩数据。使用 `os.path.join()` 构建文件路径, 确保跨平台兼容性。使用 `pd.read_excel()` 读取第一个工作表, 然后定义六门课程的列名列表, 用于后续循环处理。

2. 从 Excel 文件加载数据

```
1 # 从 Excel 文件加载数据
2 excel_path = os.path.join(os.path.dirname(__file__), 'data', '八年级期末考试成绩表.xlsx')
3 df = pd.read_excel(excel_path, sheet_name=0)
4
5 target_columns = [
6     '语文分数', '数学分数', '英语分数',
7     '物理分数', '生物分数', '地理分数'
8 ]
```

分段设置阶段，程序使用 `np.arange()` 创建从 0 到 120 的数组，间隔为 10，作为直方图的分段边界。程序创建自定义分段标签列表，前11个标签为区间格式（如 '0-9', '10-19'），最后一个标签为 '110+'，表示 110 分及以上的分数段。

3. 设置分段

```
1 # 根据数据最大值 (116) 设置分段：每10分一个分段，从0到120
2 bins = np.arange(0, 121, 10)
3 # 自定义分段标签
4 bin_labels = [f'{i}-{i+9}' for i in range(0, 110, 10)] + ['110+']
```

子图布局创建阶段，程序使用 `plt.subplots()` 创建 2×3 的子图矩阵，设置图表尺寸为 15×10 英寸。使用 `axes.flatten()` 将二维数组展平为一维数组，便于在循环中使用索引访问每个子图。

4. 创建子图布局

```
1 # 设置 2x3 子图布局
2 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
3 axes = axes.flatten()
```

直方图绘制阶段，程序遍历六门课程，为每门课程绘制一个直方图。使用 `ax.hist()` 方法绘制直方图，传入该课程的成绩数据、分段边界、边缘颜色、透明度和颜色。方法返回三个值：`counts`（每个分段的学生人数）、`edges`（分段边界）、`patches`（柱状图对象列表）。这些返回值用于后续设置刻度和添加数据标签。

5. 循环绘制直方图

```
1 # 循环绘制每个子图
2 for i, col in enumerate(target_columns):
3     ax = axes[i]
4
5     # 绘制直方图
6     counts, edges, patches = ax.hist(
7         df[col],
8         bins=bins,
9         edgecolor='black',
10        alpha=0.7,
11        color='#1f77b4', # Matplotlib默认蓝色
12        label='学生人数'
13    )
```

标题和坐标轴标签设置阶段，程序从列名中提取课程名称（去除"分数"后缀），设置子图标题包含课程名称和完整列名。设置 X 轴标签为"分数分段"，Y 轴标签为"学生人数"。

6. 设置标题和坐标轴标签

```
1 # 设置标题、坐标轴标签
2 subject_name = col.replace('分数', '')
3 ax.set_title(f'{subject_name}成绩分段统计 ({col})', fontsize=14)
4 ax.set_xlabel('分数分段', fontsize=12)
5 ax.set_ylabel('学生人数', fontsize=12)
```

刻度标签设置阶段，程序设置 X 轴刻度位置为每个分段的中心（`bins[:-1] + 5`），使用自定义的分段标签，并将标签旋转 45 度，避免重叠。程序计算每个分段的最大学生人数，设置 Y 轴范围从 0 开始，上限为最大值的 1.1 倍，为数据标签留出空间。

7. 设置刻度标签

```
1 # 设置 X 轴刻度及标签
2 ax.set_xticks(bins[:-1] + 5)
3 ax.set_xticklabels(bin_labels, rotation=45, ha='right')
4
5 # 设置 Y 轴刻度，确保从0开始
6 max_count = counts.max()
7 ax.set_ylim(0, max_count * 1.1)
```

网格线、图例和数据标签添加阶段，程序在 Y 轴方向添加虚线网格，便于读取数值。添加图例显示"学生人数"标签。程序遍历每个柱状图对象，获取其高度，如果高度大于 0，则在柱子顶部中心位置添加文本标签，显示具体的学生人数。

8. 添加网格线、图例和数据标签

```
1 # 添加网格线和图例
2 ax.grid(axis='y', linestyle='--', alpha=0.6)
3 ax.legend(loc='upper right', fontsize=10)
4
5 # 添加数据标签
6 for rect in patches:
7     height = rect.get_height()
8     if height > 0:
9         ax.text(rect.get_x() + rect.get_width() / 2., height + 0.5,
10                f'{int(height)}',
11                ha='center', va='bottom', fontsize=9)
```

最后，程序使用 `plt.tight_layout()` 调整布局，防止标签重叠。保存图表为 PNG 格式，设置分辨率为 300 DPI，最后显示图表。

9. 保存和显示图形

```
1 # 调整布局以防止重叠
2 plt.tight_layout()
3
4 # 保存图形
5 plt.savefig('grade_histogram.png', dpi=300, bbox_inches='tight')
6 print('图表已保存为 grade_histogram.png')
7
8 # 显示图形
9 plt.show()
```

4.4 执行方法和运行结果

4.4.1 执行方法

1. 环境要求：

- Python 3.x
- matplotlib 库
- numpy 库
- pandas 库
- openpyxl 库（用于读取 Excel 文件）

2. 运行步骤：

```
1 # 进入项目目录
2 cd task3/task3-4
3
4 # 运行程序
5 python main.py
```

3. 依赖安装：

```
1 pip install matplotlib numpy pandas openpyxl
```

或使用项目中的 `uv` 包管理器：

```
1 uv sync
```

4.4.2 运行结果

程序运行后会弹出窗口显示生成的成绩分段统计直方图，同时保存在本地，如图 4-1 所示：

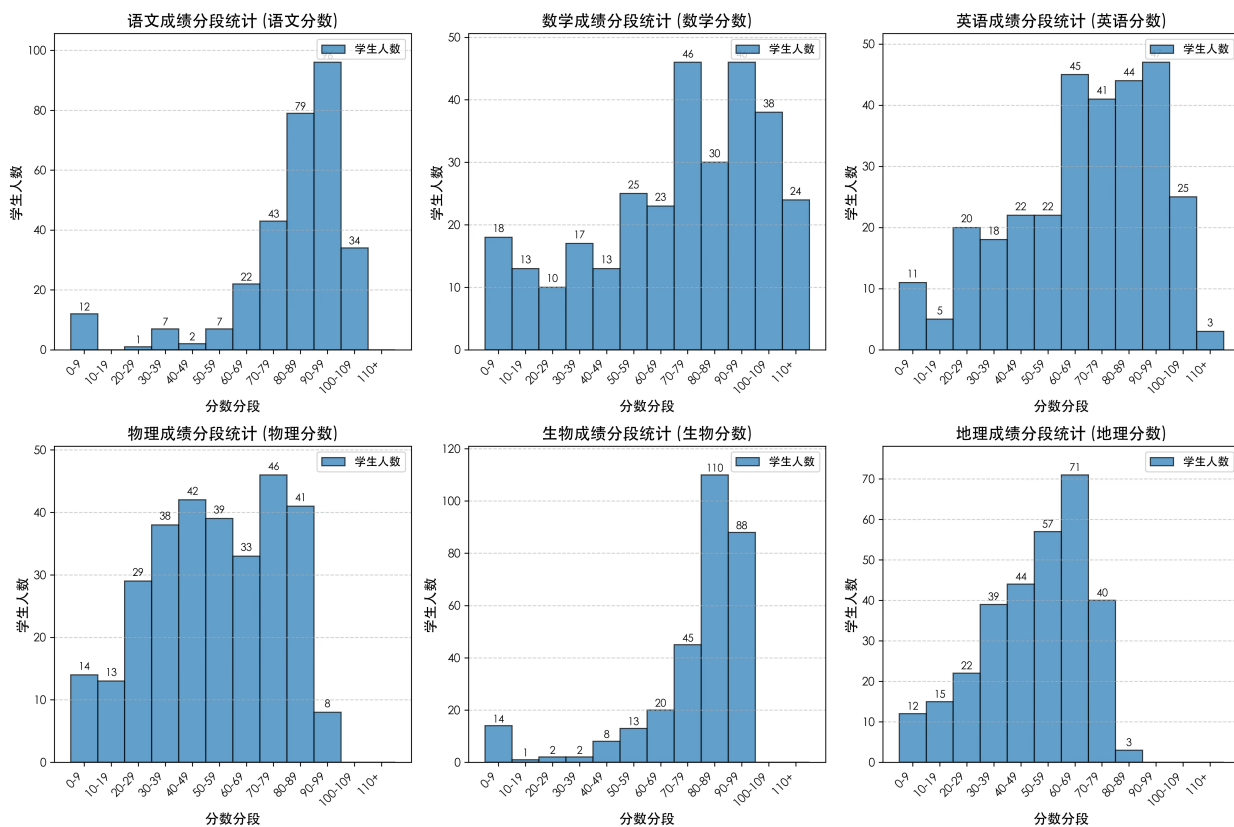


图 4-1 任务四运行结果

图表正确包含以下特征：

1. 图表结构：

- 2×3 的子图布局，共 6 个子图
- 每个子图对应一门课程（语文、数学、英语、物理、生物、地理）

2. 直方图展示：

- 每个子图显示该课程的成绩分段分布
- 分段为每10分一个区间（0-9, 10-19, 20-29, ..., 110+）
- 柱状图高度表示该分段的学生人数

3. 坐标轴标签：

- X轴：显示分数分段标签，旋转45度便于阅读
- Y轴：显示学生人数，从0开始

4. 数据标签：

- 每个柱子上方显示具体的学生人数
- 便于精确读取各分段的人数

5. 图例和网格线：

- 每个子图右上角显示"学生人数"图例
- Y轴方向添加虚线网格，便于读取数值

生成的图表完整、清晰地展示了六门课程的成绩分段统计情况，符合作业要求的所有绘图元素标准。

5 任务五

5.1 作业要求

使用 `BeijingPM20100101_20151231.csv` 数据集，展示北京市2010~2015年 PM2.5 指数月平均数据的变化情况，在同一幅图中绘制六条折线，每年一条折线。其他要求同任务一。

5.2 图表分析

通过分析任务要求和数据集，可以得出以下关键信息：

1. **图表类型：**折线图（Line Chart），用于展示时间序列数据的变化趋势。

2. **数据特征：**

- 数据来源：BeijingPM20100101_20151231.csv 数据集
- 时间范围：2010-2015年（共6年）
- 数据维度：PM2.5指数月平均值
- 站点选择：使用四个PM2.5监测站点的平均值（PM_Dongsi, PM_Dongsihuan, PM_Nongzhanguan, PM_US Post）

3. **数据处理：**

- 缺失值处理：这里按照作业二的思路处理缺失值
 - PM2.5相关列：线性插值
 - 气象数据：线性插值
 - 风向：前向/后向填充
 - 降水量：填充为0
- 数据聚合：按年份和月份分组，计算月平均值
- 站点平均：对四个站点的PM2.5值求平均，提高数据可靠性

4. **图表结构：**

- 单图展示，包含6条折线
- 每条折线对应一个年份（2010-2015）
- X轴：月份（1-12月）
- Y轴：PM2.5指数月平均值

5. **图表元素：**

- 标题：显示"北京市2010-2015年PM2.5指数月平均数据变化"
- 坐标轴标签：X轴标注"月份"，Y轴标注"PM2.5指数月平均值"
- 刻度标签：X轴显示"1月"到"12月"，Y轴显示数值
- 图例：显示各年份，便于区分不同折线
- 网格线：添加网格线，便于读取数值
- 标记点：每条折线上有标记点，突出数据点位置

5.3 程序设计

5.3.1 设计思路

本程序使用 Python 的 matplotlib 库绘制PM2.5指数月平均数据折线图，主要设计思路如下：

1. **数据加载**：从CSV文件加载PM2.5数据
2. **缺失值处理**：按照标准流程处理各类缺失值
3. **站点选择**：选择三个PM2.5监测站点并计算平均值
4. **数据筛选**：筛选2010-2015年的数据
5. **数据聚合**：按年份和月份分组，计算月平均值
6. **数据重塑**：将数据重塑为适合绘图的格式
7. **折线图绘制**：绘制每年的折线图
8. **样式设置**：设置标题、坐标轴标签、图例等

5.3.2 核心代码

程序首先导入必要的库并设置中文字体支持。数据读取阶段，程序从 CSV 文件加载 PM2.5 数据集，该数据集包含2010-2015年的逐小时观测数据。

1. 字体和基础配置

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3 import numpy as np
4 import matplotlib as mpl
5 import os
6
7 # 设置中文字体
8 mpl.rcParams['font.family'] = 'STHeiti'
9 mpl.rcParams['axes.unicode_minus'] = False
```

数据读取阶段，程序使用 `os.path.join()` 构建 CSV 文件路径，使用 `pd.read_csv()` 读取数据。该数据集包含多个 PM2.5 监测站点的数据以及气象数据。

2. 读取数据

```
1 # 读取数据
2 csv_path = os.path.join(os.path.dirname(__file__), 'data',
3                           'BeijingPM20100101_20151231.csv')
4 df = pd.read_csv(csv_path)
```

缺失值处理阶段，程序按照作业二的思路对不同类型的数据采用不同的处理方法。对于 PM2.5 相关列，使用线性插值方法填充缺失值，因为 PM2.5 浓度是连续变化的物理量，相邻时刻的值存在强相关性。`limit_direction='both'` 参数表示同时向前和向后插值，确保序列开头和结尾的缺失值也能被填充。对于气象数据（温度、湿度、气压、风速等），同样使用线性插值。对于风向数据，使用前向填充（`ffill()`）和后向填充（`bfill()`）相结合的方法，因为风向是分类变量，不适合插值。对于降水量数据，将缺失值填充为 0，因为无降水时通常不记录数值。

3. 处理缺失值

```
1 # 按照 process_pm25.py 的逻辑处理缺失值
2 df_processed = df.copy()
3
4 # PM2.5相关列 - 线性插值
5 pm_columns = ['PM_Dongsi', 'PM_Dongsihuan', 'PM_Nongzhanguan', 'PM_US Post']
6 for col in pm_columns:
7     if col in df_processed.columns:
8         df_processed[col] = df_processed[col].interpolate(method='linear',
9                                                             limit_direction='both')
10
11 # 气象数据 - 线性插值
12 weather_columns = ['DEWP', 'TEMP', 'HUMI', 'PRES', 'Iws']
13 for col in weather_columns:
14     if col in df_processed.columns:
15         df_processed[col] = df_processed[col].interpolate(method='linear',
16                                                             limit_direction='both')
17
18 # 风向 - 前向填充
19 if 'cbwd' in df_processed.columns:
20     df_processed['cbwd'] = df_processed['cbwd'].ffill()
21     df_processed['cbwd'] = df_processed['cbwd'].bfill()
22
23 # 降水量 - 填充为0
24 precipitation_columns = ['precipitation', 'Iprec']
25 for col in precipitation_columns:
26     if col in df_processed.columns:
27         df_processed[col] = df_processed[col].fillna(0)
```

站点选择和平均值计算阶段，程序定义四个 PM2.5 监测站点的列名，检查这些列是否存在于 DataFrame 中。使用所有可用的站点数据，计算四个站点的平均值作为 PM2.5 指数。这种方法能够提高数据的可靠性，减少单个站点数据异常对结果的影响。

4. 选择四个站点并求平均

```
1 # 选择四个PM2.5站点并求平均
2 selected_pm_columns = ['PM_Dongsi', 'PM_Dongsihuan', 'PM_Nongzhanguan', 'PM_US
  Post']
3 available_pm_columns = [col for col in selected_pm_columns if col in
  df_processed.columns]
4
5 # 使用所有可用的PM2.5站点
6 pm_columns_to_use = available_pm_columns
7
8 # 计算四个站点的平均值
9 df_processed['PM_avg'] = df_processed[pm_columns_to_use].mean(axis=1)
```

数据筛选和聚合阶段，程序首先筛选出 2010-2015 年的数据，然后使用 `groupby()` 方法按年份和月份分组，计算每组的 PM2.5 平均值。
`reset_index()` 方法将分组后的索引转换为普通列，便于后续处理。

5. 筛选数据并计算月平均值

```
1 # 筛选 2010-2015 年数据并计算月平均值
2 df_filtered = df_processed[(df_processed['year'] >= 2010) &
  (df_processed['year'] <= 2015)].copy()
3 monthly_avg = df_filtered.groupby(['year', 'month'])
  ['PM_avg'].mean().reset_index()
```

数据重塑阶段，程序使用 `pivot_table()` 方法将数据重塑为透视表格式。将月份作为索引（行），年份作为列，PM2.5 平均值作为值。这种格式便于在循环中为每年绘制一条折线。

6. 数据重塑

```
1 # 数据重塑：将年份作为列，月份作为索引
2 pm_pivot = monthly_avg.pivot_table(
3     index='month',
4     columns='year',
5     values='PM_avg'
6 )
```

折线图绘制阶段，程序创建 12×7 英寸的图表对象。使用 `plt.cm.viridis` 颜色映射生成颜色列表，确保不同年份的折线使用不同颜色。程序遍历每年的数据，使用 `ax.plot()` 方法绘制折线图，设置标记点为圆形，线宽为 2，标记点大小为 6，确保图表清晰可见。

7. 绘制折线图

```
1 # 绘图
2 fig, ax = plt.subplots(figsize=(12, 7))
3
4 # 绘制每年的折线图
5 colors = plt.cm.viridis(np.linspace(0, 1, len(pm_pivot.columns)))
6 for i, year in enumerate(pm_pivot.columns):
7     ax.plot(
8         pm_pivot.index,
9         pm_pivot[year],
10        label=str(year),
11        marker='o',
12        linestyle='--',
13        color=colors[i],
14        linewidth=2,
15        markersize=6
16    )
```

标题、坐标轴标签和图例设置阶段，程序设置图表标题，包含时间范围和数据类型说明。设置 X 轴标签为“月份”，Y 轴标签说明使用四个站点的平均值。程序设置 X 轴刻度为 1-12 月，并将标签格式化为“1月”、“2月”等形式。添加虚线网格，便于读取数值。添加图例，显示各年份，便于区分不同折线。设置 Y 轴从 0 开始，确保数据对比的准确性。

8. 设置标题、坐标轴标签和图例

```
1 # 添加标题、坐标轴标签和图例
2 site_names = ', '.join(pm_columns_to_use)
3 ax.set_title('北京市2010-2015年PM2.5指数月平均数据变化', fontsize=16,
4             fontweight='bold')
5 ax.set_xlabel('月份', fontsize=12)
6 ax.set_ylabel(f'PM2.5指数月平均值 (四个站点平均)', fontsize=12)
7
8 # 设置 x 轴刻度标签为 '1月', '2月' ...
9 ax.set_xticks(pm_pivot.index)
10 ax.set_xticklabels([f'{m}月' for m in pm_pivot.index])
11
12 # 添加网格线和图例
13 ax.grid(True, linestyle='--', alpha=0.6)
14 ax.legend(title='年份', loc='upper right')
15
16 # 设置 y 轴从 0 开始
17 ax.set_ylim(bottom=0)
```

最后，程序调整布局，保存图表为 PNG 格式，设置分辨率为 300 DPI，并输出使用的站点信息，最后显示图表。

9. 保存和显示图形

```
1 # 调整布局
2 plt.tight_layout()
3
4 # 保存图形
5 plt.savefig('pm25_monthly_avg.png', dpi=300, bbox_inches='tight')
6 print(f'图表已保存为 pm25_monthly_avg.png')
7 print(f'使用的PM2.5站点: {"", ".join(pm_columns_to_use)}')
8
9 # 显示图形
10 plt.show()
```

5.4 执行方法和运行结果

5.4.1 执行方法

1. 环境要求：

- Python 3.x
- matplotlib 库
- numpy 库
- pandas 库

2. 运行步骤：

```
1 # 进入项目目录
2 cd task3/task3-5
3
4 # 运行程序
5 python main.py
```

3. 依赖安装：

```
1 pip install matplotlib numpy pandas
```

或使用项目中的 `uv` 包管理器：

```
1 uv sync
```

5.4.2 运行结果

程序运行后会弹出窗口显示生成的PM2.5指数月平均数据折线图，同时保存在本地，如图 5-1 所示：

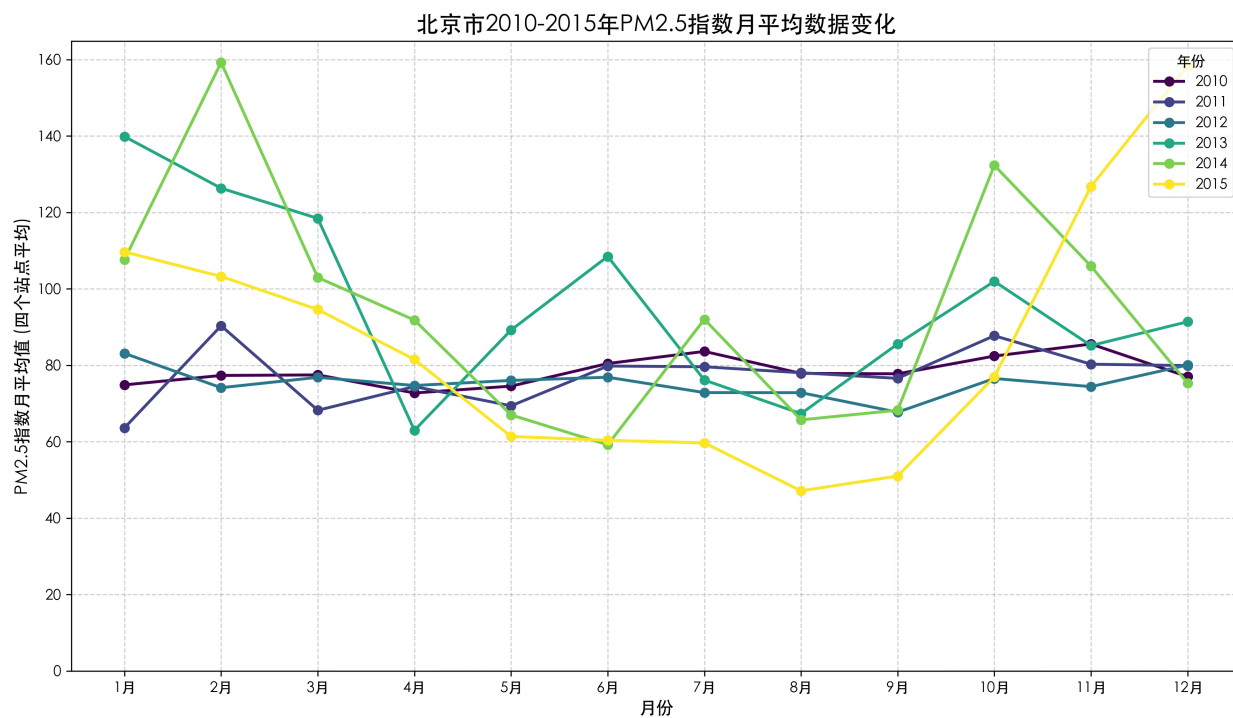


图 5-1 任务五运行结果

图表正确包含以下特征：

1. 图表结构：

- 单图展示，包含6条折线
- 每条折线对应一个年份（2010-2015）

2. 折线图展示：

- 每条折线显示该年份的PM2.5指数月度变化
- X轴：1-12月
- Y轴：PM2.5指数月平均值（四个站点平均）
- 每条折线上有标记点，便于识别数据点

3. 坐标轴标签：

- X轴：显示"1月"到"12月"
- Y轴：显示PM2.5指数月平均值，从0开始

4. 图例和网格线：

- 右上角显示年份图例，便于区分不同折线
- 添加虚线网格，便于读取数值

生成的图表完整、清晰地展示了北京市2010-2015年PM2.5指数月平均数据的变化情况，符合作业要求的所有绘图元素标准。

6 作业总结与心得体会

本次作业涵盖了数据可视化的核心环节，让我对数据从加载到可视化的完整流程有了更深入的理解。在完成五个任务的过程中，我不仅掌握了 matplotlib 绘图库的基本使用方法，更重要的是深入体会到了数据可视化在整个数据分析流程中的关键作用。通过绘制柱状图、饼图、散点图矩阵、直方图和折线图等多种图表类型，我深刻认识到选择合适的可视化方法对于有效传达数据信息的重要性。

通过本次作业，我深刻认识到"一图胜千言"这一数据可视化的基本原则。数据可视化不仅仅是简单地将数据转换为图形，更重要的是通过合理的图表设计，让数据背后的信息能够清晰、直观地传达给读者。同时，matplotlib 库的强大功能为不同类型的图表绘制提供了便捷的解决方案。通过本次作业，我不仅掌握了 matplotlib 的基本绘图方法，更重要的是理解了图表元素（标题、坐标轴标签、刻度标签、图例等）对图表可读性的重要性。

综上，本次作业让我深刻认识到，数据可视化是数据分析流程中不可或缺的关键环节。高质量的数据可视化不仅能够提高分析结果的可读性，还能为后续的数据分析和决策工作提供有力支持。在未来的学习和工作中，我将继续深入探索数据可视化的各种方法和技术，不断提升数据分析和展示的能力。

7 附录A：提交材料清单和说明

本次作业提交材料的目录结构如下：

1	.	
2	├ docs/	
3	│ └ report.md	# 本次作业报告 (Markdown 源文件)
4	├ task3-1/	# 任务一：人口普查柱状图
5	│ └ main.py	# 绘制历次人口普查全国人口数量柱状图的核心代码
6	│ └ population_census.png	# 任务一生成的柱状图图片文件
7	│ └ pyproject.toml	# 任务一 Python 项目配置 (uv 包管理)
8	│ └ uv.lock	# 任务一依赖锁定文件，保证环境可重现
9	├ task3-2/	# 任务二：支付宝支出饼图
10	│ └ main.py	# 绘制 2020 年支付宝年度支出分类占比饼图的核心代码
11	│ └ expense_pie_chart.png	# 任务二生成的饼图图片文件
12	│ └ pyproject.toml	# 任务二 Python 项目配置
13	│ └ uv.lock	# 任务二依赖锁定文件
14	├ task3-3/	# 任务三：IRIS 数据集散点图矩阵
15	│ └ data/	
16	│ │ └ iris.csv	# IRIS 数据集 CSV 文件 (作业提供的数据)
17	│ └ main.py	# 绘制 4×4 散点图矩阵的核心代码 (从 CSV 加载数据)
18	│ └ iris_scatter_matrix.png	# 任务三生成的散点图矩阵图片文件
19	│ └ pyproject.toml	# 任务三 Python 项目配置
20	│ └ uv.lock	# 任务三依赖锁定文件
21	├ task3-4/	# 任务四：成绩分段统计直方图
22	│ └ data/	
23	│ │ └ 八年级期末考试成绩表.xlsx	# 八年级期末考试原始成绩表 (Excel 文件)
24	│ └ main.py	# 绘制六门课程成绩分段直方图的核心代码
25	│ └ grade_histogram.png	# 任务四生成的 2×3 直方图图片文件
26	│ └ pyproject.toml	# 任务四 Python 项目配置
27	│ └ uv.lock	# 任务四依赖锁定文件
28	├ task3-5/	# 任务五：PM2.5 指数月平均折线图
29	│ └ data/	
30	│ │ └ BeijingPM20100101_20151231.csv	# 北京 PM2.5 原始数据集 (2010-2015, 逐小时)
31	│ └ main.py	# 按四个站点缺失值处理并绘制 2010-2015 月均折线图的核心代码
32	│ └ pm25_monthly_avg.png	# 任务五生成的 PM2.5 月平均折线图图片文件
33	│ └ pyproject.toml	# 任务五 Python 项目配置
34	│ └ uv.lock	# 任务五依赖锁定文件